

PL/SQL PACKAGE

◇ Definition of Package PL/SQL

A package is a collection of related procedures, functions, variables, cursors, and exceptions stored together as a single unit in the database.

A package is a **schema object** that contains a set of **public declarations** (package specification) and **private implementations** (package body). It allows developers to define an interface that can be accessed by users while hiding the internal logic.

◇ Parts of a Package

A PL/SQL package consists of **two main parts**:

1. Package Specification

The **package specification** is the **declaration part** of the package. It defines:

- Procedures
- Functions
- Variables
- Cursors
- Exceptions

These declared members are **visible to users** of the package. The specification acts as a **contract** between the package and the calling programs.

2. Package Body

The **package body** contains the **implementation** of procedures and functions declared in the specification. It also allows declaration of **private elements** that are not accessible outside the package. The package body is optional, but it is required if the specification contains procedures or functions.

◇ Advantages of Packages

Packages provide several important benefits:

1. **Modularity** – Related program units are grouped together.
2. **Better Performance** – Once loaded into memory, the package remains there, reducing repeated loading.
3. **Encapsulation** – Internal logic can be hidden from users.
4. **Easy Maintenance** – Changes can be made in the body without affecting users.
5. **Reusability** – Same package can be used by multiple applications.
6. **Security** – Access can be granted to the package instead of individual objects.

◇ Features of PL/SQL Package

- Supports **overloading** (same procedure name with different parameters)
- Allows **global variables**
- Improves **code readability**
- Reduces **network traffic**
- Enables **team development**

◇ Syntax of Package

Package Specification

```
CREATE OR REPLACE PACKAGE package_name AS  
    procedure_name;  
    function_name;  
END package_name;  
/
```

Package Body

```
CREATE OR REPLACE PACKAGE BODY package_name AS  
    procedure_name IS  
    BEGIN  
        statements;  
    END;
```

```

function_name RETURN datatype IS
BEGIN
    statements;
    RETURN value;
END;
END package_name;
/

```

♦ Practical Example

A package can be created for an **EMP table** to perform operations such as insert, update, and retrieve salary. The package specification declares these operations, and the package body implements them. Users can call these procedures and functions without knowing the internal SQL logic.

Practical Example – EMP Table

```

CREATE TABLE emp (
    empno  NUMBER(4),
    ename  VARCHAR2(30),
    job    VARCHAR2(20),
    salary NUMBER(7,2),
    deptno NUMBER(2),
    city   VARCHAR2(20)
);

```

Package Specification Example

```

CREATE OR REPLACE PACKAGE emp_pkg AS

PROCEDURE add_emp(
    p_empno emp.empno%TYPE,
    p_ename emp.ename%TYPE,
    p_job emp.job%TYPE,
    p_salary emp.salary%TYPE,
    p_deptno emp.deptno%TYPE,
    p_city emp.city%TYPE
);

```

```
FUNCTION get_salary(  
    p_empno emp.empno%TYPE  
    ) RETURN NUMBER;  
END emp_pkg;  
/
```

Package Body Example

CREATE OR REPLACE PACKAGE BODY emp_pkg AS

```
PROCEDURE add_emp(  
    p_empno emp.empno%TYPE,  
    p_ename emp.ename%TYPE,  
    p_job emp.job%TYPE,  
    p_salary emp.salary%TYPE,  
    p_deptno emp.deptno%TYPE,  
    p_city emp.city%TYPE  
    ) IS  
BEGIN  
    INSERT INTO emp VALUES  
        (p_empno, p_ename, p_job, p_salary, p_deptno, p_city);  
END add_emp;
```

```
FUNCTION get_salary(  
    p_empno emp.empno%TYPE  
    ) RETURN NUMBER IS  
    v_salary emp.salary%TYPE;  
BEGIN  
    SELECT salary INTO v_salary FROM emp WHERE empno = p_empno;  
    RETURN v_salary;  
END get_salary;  
END emp_pkg;  
/
```

Execution of Package

BEGIN

emp_pkg.add_emp(101,'Ramesh','Clerk',25000,20,'Pune');

END;

/

SET SERVEROUTPUT ON;

DECLARE

s NUMBER;

BEGIN

s := emp_pkg.get_salary(101);

DBMS_OUTPUT.PUT_LINE('Salary = ' || s);

END;

/

◇ Difference Between Package and Procedure

Package	Procedure
Collection of program units	Single program unit
Has specification & body	Has only one block
Supports encapsulation	No encapsulation
Improves performance	Limited performance benefit